



**HOGESCHOOL
UTRECHT**



ACHTERHALEN WACHTWOORD

Naam: Ali Othman
Student nummer : 1860574

April 24, 2025
P2_S1_Achterhalen wachtwoord



Naam: Ali Othman
Student nummer : 1860574

April 24, 2025
P2_S1_Achterhalen wachtwoord

Index

1. Inleiding	4
◦ Doel van het document	
◦ Belang van wachtwoordbeveiliging	
2. Hashing: Theorie en Basisprincipes	4
◦ Wat is hashing?	
◦ Belangrijke kenmerken van hashing	
◦ Toepassingen in cybersecurity	
3. Hashing versus Encryptie	5-6
◦ Fundamentele verschillen	
◦ Gebruiksscenario's voor beide technieken	
4. Overzicht van Hashing-algoritmen	
4.1 MD5 (Message Digest Algorithm 5)	6-8
◦ Werking en structuur	
◦ Beveiligingsrisico's en beperkingen	
4.2 SHA-1 (Secure Hash Algorithm 1)	9-11
◦ Technische achtergrond	
◦ Redenen voor veroudering	
4.3 SHA-256 (Secure Hash Algorithm 256-bit)	11-13
◦ Sterkte en moderne toepassingen	
◦ Gebruik in blockchain en SSL	
4.4 bcrypt	14-15
◦ Salt-mechanisme en kostenfactor	
◦ Voordelen voor wachtwoordopslag	

4.5 Argon2	16-17
◦ Geheugengebruik en aanpasbaarheid	
◦ Varianten (Argon2d, Argon2i, Argon2id)	
4.6 Vergelijkingstabel: Hashing-algoritmen	18
◦ Veiligheid, snelheid en aanbevelingen	
5.Wachtwoordkraken: Methoden en Technieken	20-21
◦ Brute-force-aanvallen	
◦ Woordenlijstaanvallen (dictionary attacks)	
◦ Rainbow table-aanvallen	
◦ Social engineering en phishing	
6.Praktijk: Tools voor Wachtwoordkraken in Kali Linux	
6.1 John the Ripper	22-25
◦ Introductie	
◦ Voorbeeldcommando's en use-cases	
◦ Uitdagingen	
◦ Conclusie	
6.2 Hashcat	26-30
◦ Introductie	
◦ Voorbeeldcommando's en use-cases	
◦ Uitdagingen	
◦ Conclusie	
6.3 Proces Infographic	31
7.Beveiligingsmaatregelen en Best Practices	32
◦ Aanmaken van sterke wachtwoorden	
◦ Gebruik van wachtwoordmanagers	
◦ Tweefactorauthenticatie (2FA)	
◦ Regelmatige updates en audits	
8.Bronnen en Referenties	33
◦ Technische documentatie	
◦ Aanbevolen literatuur en websites	

Inleiding

In een wereld waarin digitale veiligheid steeds belangrijker wordt, speelt wachtwoordbeveiliging een cruciale rol in het beschermen van gevoelige informatie. Toch blijken wachtwoorden in de praktijk vaak een zwakke schakel. Deze opdracht richt zich op het onderzoeken van hoe wachtwoorden worden versleuteld en op welke manieren deze in sommige gevallen alsnog kunnen worden achterhaald. Het doel is om inzicht te verkrijgen in verschillende methoden van wachtwoordversleuteling en wachtwoordaanvallen. Aan de hand van praktijkgerichte oefeningen wordt onderzocht hoe wachtwoorden worden beveiligd met technieken zoals MD5, en hoe deze beveiligingen eventueel doorbroken kunnen worden, bijvoorbeeld via brute-force-aanvallen.

Wat is Hashing?

Hashing is een techniek in de digitale wereld die wordt gebruikt om gegevens, zoals wachtwoorden, te beveiligen. Met hashing wordt een wachtwoord omgezet in een reeks willekeurige tekens, ook wel een "hash" genoemd. Dit proces is éénrichtingsverkeer: je kunt de originele tekst niet zomaar terughalen uit de hash. Zo kunnen wachtwoorden veilig opgeslagen worden, zelfs als iemand toegang krijgt tot de database. Verschillende algoritmes zoals MD5, SHA-1 en bcrypt zorgen voor deze omzetting. Hashing helpt dus om gevoelige informatie te beschermen tegen hackers.

Belangrijke kenmerken van hashing:

- Vast formaat: Ongeacht hoe groot de invoer is, de hash heeft altijd dezelfde lengte (bijv. SHA-256 geeft altijd 64 tekens).
- Snel & efficiënt: Het is gemakkelijk om van data een hash te maken, maar bijna onmogelijk om de oorspronkelijke data terug te halen uit de hash.
- Uniek (idealiter): Kleine veranderingen in de invoer geven een compleet andere hash.
- Veiligheidsdoel: Wordt vaak gebruikt om wachtwoorden op te slaan (zonder het origineel te bewaren) of om data-integriteit te controleren.

Voorbeeld:

Invoer: "hallo" → Hash (MD5): 598d4c200461b81522a3328565c25f7c

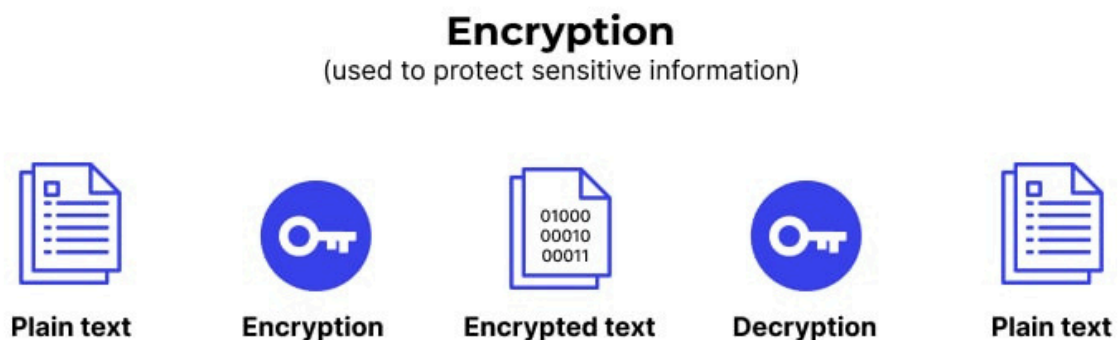
Invoer: "hallo1" → Hash (MD5): 8f0f6d5f0e016e4f70a0a7a8822f2e3c

Toepassingen:

- Wachtwoordopslag (bv. websites slaan alleen hashes op).
- Bestandscontrole (bv. checken of een download niet is aangepast).
- Blockchain (transacties verifiëren).

Hashing vs Encryptie : Wat is het Verschil?

Naast hashing heb je ook encryptie, een andere manier om gegevens te beveiligen. Bij encryptie worden gegevens omgezet in onleesbare tekens, zodat anderen ze niet zomaar kunnen begrijpen. Alleen met een speciale sleutel kun je de originele informatie weer leesbaar maken. Encryptie wordt vaak gebruikt om e-mails, bestanden of netwerkverkeer veilig te versturen, zodat kwaadwillenden er geen toegang tot krijgen.



Belangrijkste Verschillen: Hashing

Encryptie

Niet omkeerbaar	Omkeerbaar (met sleutel)
Gebruikt voor verificatie	Gebruikt voor vertrouwelijke communicatie
Geen sleutels nodig	Vereist een encryptie-/decryptiesleutel

Overzicht van veelgebruikte hashing-algoritmes

Er zijn veel verschillende soorten hashing. In dit document worden vijf veelgebruikte vormen van hashing besproken die vaak voorkomen in cybersecurity. Elke methode heeft zijn eigen sterktes en zwaktes. Van elke soort wordt een korte uitleg gegeven, met een vergelijking op het gebied van veiligheid en snelheid. Zo ontstaat een duidelijk beeld van welke hashing-algoritmes geschikt zijn voor verschillende toepassingen.

1)MD5 (Message Digest Algorithm 5) hash-algoritme

MD5 is een veelgebruikte hashfunctie die een stukje data omzet in een unieke reeks van 32 tekens (128 bits). Het werd vroeger vaak gebruikt voor het opslaan van wachtwoorden of het controleren van bestanden. Tegenwoordig wordt MD5 als onveilig beschouwd, omdat het mogelijk is om twee verschillende invoeren met dezelfde hash te maken (dit heet een collisie). Daardoor is MD5 niet meer geschikt voor gevoelige toepassingen.

Hoe werkt het MD5-algoritme?

1.Invoer voorbereiden

De originele data wordt opgedeeld in blokken van 512 bits. Als de data korter is, wordt deze opgevuld volgens een vast patroon (padding), zodat het precies in blokken past.

2.Initialisatie

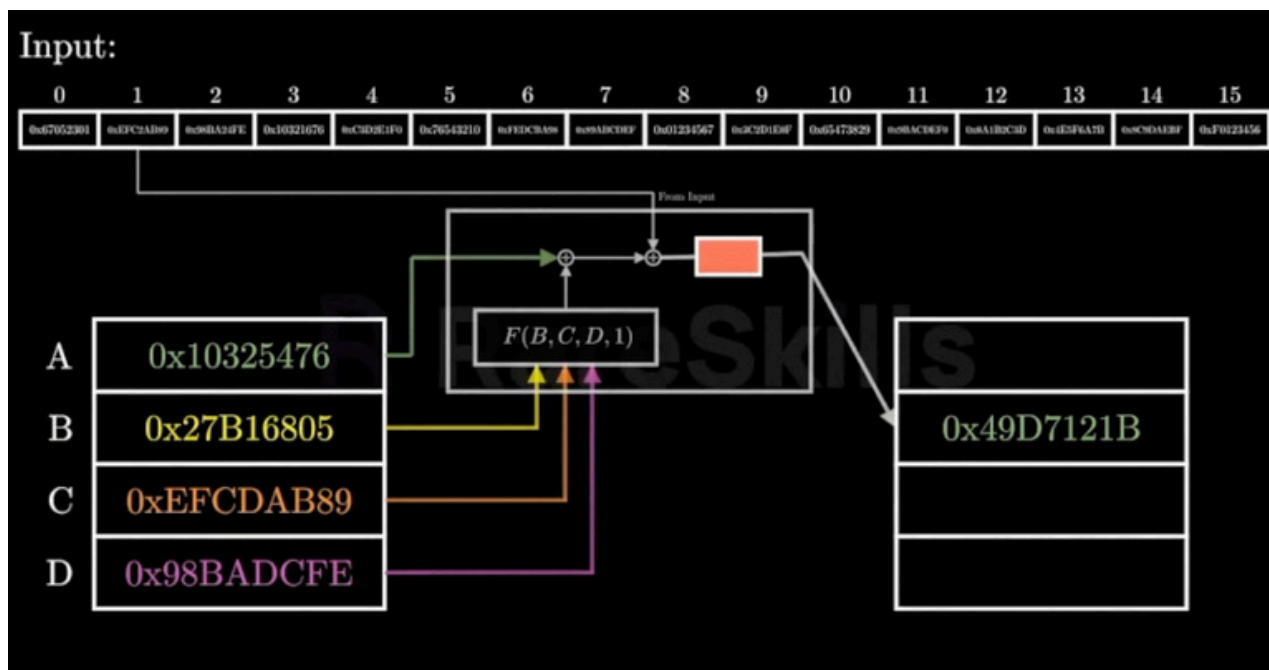
MD5 start met vier vaste waarden (A, B, C en D), die later tijdens het proces steeds worden aangepast.

3.Hoofdverwerking

Elk blok wordt verwerkt in 64 stappen. Hierbij worden logische functies (zoals AND, OR, NOT en XOR) gebruikt, samen met wiskundige bewerkingen. Deze stappen veranderen de beginwaarden (A, B, C, D).

4. Output genereren

Na het verwerken van alle blokken worden de vier waarden (A, B, C, D) gecombineerd tot één hash van 128 bits (32 tekens lang).



MD5 function foto

Voordelen en nadelen van MD5:

Voordelen van MD5

Snel en efficiënt

MD5 werkt snel en is licht in gebruik, waardoor het geschikt is voor toepassingen waar snelheid belangrijk is.

Vaste lengte output

Ongeacht de grootte van de invoer levert MD5 altijd een hash op van 128 bits (32 tekens), wat handig is voor opslag en vergelijking.

Breed ondersteund

Omdat MD5 al lang bestaat, wordt het door veel systemen, talen en tools ondersteund.

Nadelen van MD5

Niet meer veilig

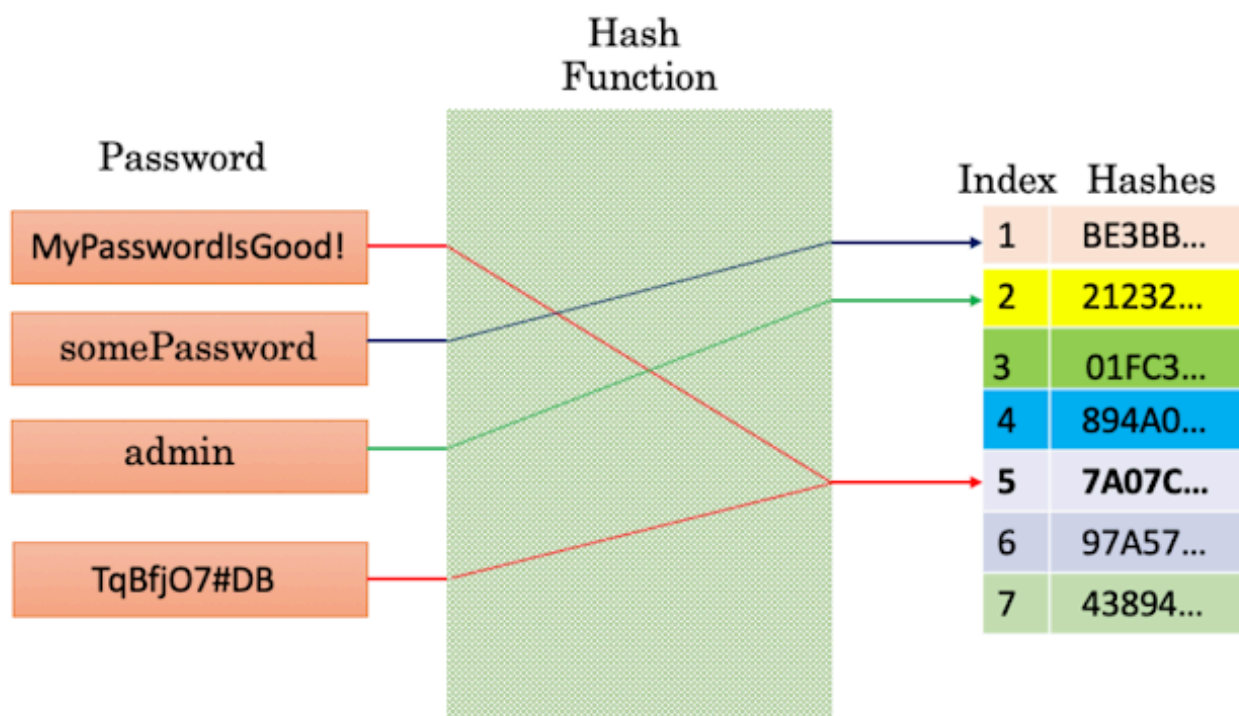
MD5 is kwetsbaar voor collisies (twee verschillende invoeren met dezelfde hash), wat het ongeschikt maakt voor beveiligingstoepassingen.

Gevoelig voor brute-force en rainbow table aanvallen

Door de snelheid van MD5 is het makkelijker voor aanvallers om veel wachtwoorden te proberen of kant-en-klare lijsten te gebruiken.

Geen salt ondersteuning van zichzelf

MD5 gebruikt standaard geen "salt" (extra willekeurige gegevens), wat het makkelijker maakt om hashes te kraken.



Hashing collisies foto

Conclusie:

MD5 is snel en handig voor eenvoudige toepassingen zoals het controleren van de integriteit van bestanden (bijvoorbeeld om te zien of een bestand onderweg beschadigd is geraakt). Voor zulke niet-kritieke doeleinden kan MD5 nog steeds bruikbaar zijn. Maar als het gaat om beveiliging van gevoelige gegevens, zoals wachtwoorden of persoonlijke informatie, is MD5 sterk verouderd en onveilig. Door de bekende kwetsbaarheden zoals collisies en het ontbreken van ingebouwde bescherming zoals salting, is het relatief eenvoudig voor hackers om MD5-hashes te kraken met moderne technieken zoals rainbow tables of brute-force aanvallen. Voor veilige toepassingen is het dus aan te raden om moderne en sterkere hash-algoritmes te gebruiken zoals SHA-256, SHA-3, of bcrypt, die speciaal zijn ontworpen om moeilijk te kraken te zijn en beter bestand zijn tegen hedendaagse aanvallen.

2)SHA-1 (Secure Hash Algorithm 1) hash-algoritme

SHA-1 werd ontwikkeld door de Amerikaanse overheid en lang werd gebruikt om gegevens te beveiligen. Het algoritme zet invoer (zoals een wachtwoord of bestand) om in een vaste reeks van 160 bits (40 hexadecimale tekens). Net als bij andere hashfuncties is het doel van SHA-1 om unieke "vingerafdrukken" van data te maken, zodat je kunt controleren of de gegevens zijn veranderd. SHA-1 was jarenlang populair voor digitale handtekeningen, SSL-certificaten en het controleren van bestanden. Maar door nieuwe technieken is aangetoond dat ook SHA-1 kwetsbaar is voor collisies, waarbij twee verschillende invoeren dezelfde hash opleveren. Hierdoor wordt SHA-1 tegenwoordig als onveilig beschouwd voor gevoelige toepassingen.

Hoe werkt SHA-1?

Vorbereiding van de invoer

De originele data wordt eerst verlengd (gepad) zodat de lengte een veelvoud is van 512 bits. Er wordt ook informatie over de lengte van de originele data toegevoegd.

Verdeling in blokken

De gepadde invoer wordt opgedeeld in blokken van 512 bits.

Initialisatie van variabelen

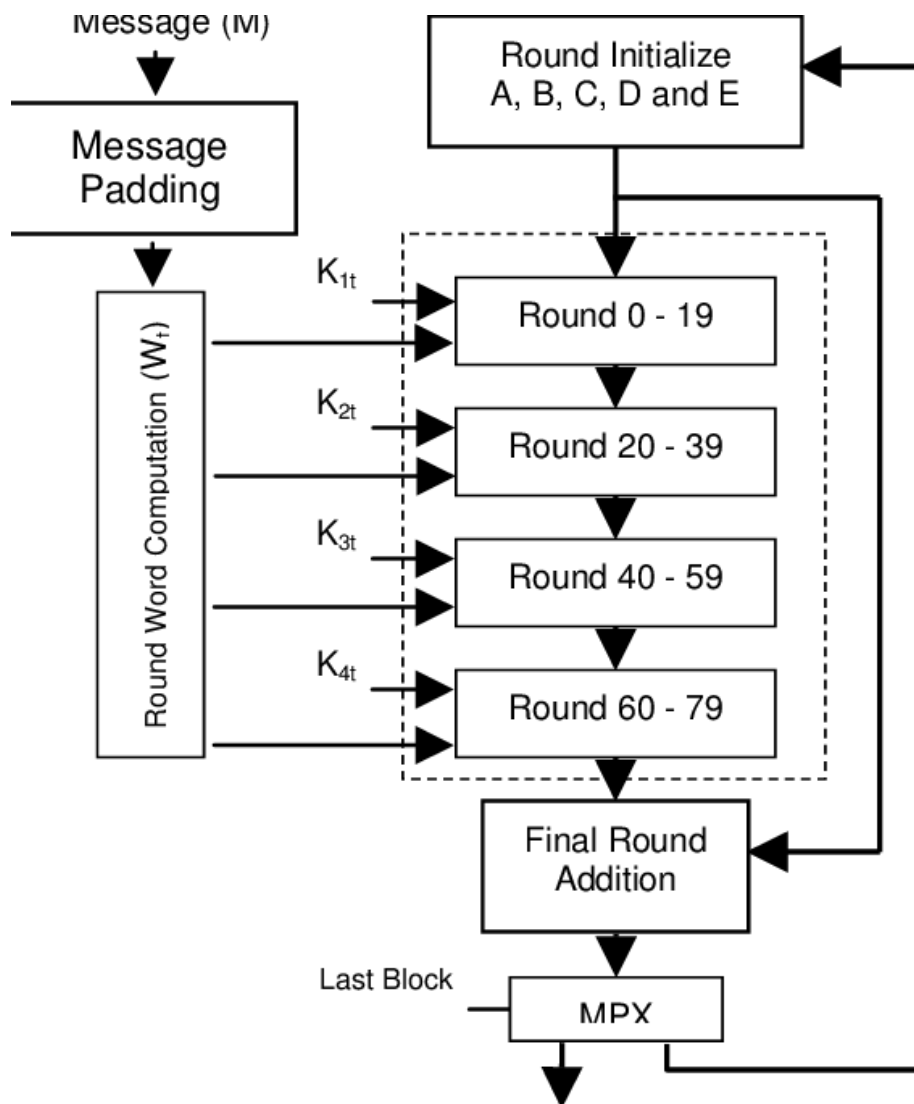
SHA-1 gebruikt vijf vaste beginwaarden (A, B, C, D en E), elk 32 bits lang. Deze worden telkens bijgewerkt in het proces.

Hoofdverwerking (80 rondes)

Elk blok wordt verwerkt in 80 rondes van bewerkingen. In elke ronde worden logische functies (zoals AND, OR, XOR) en bitverschuivingen gebruikt om de waarden A t/m E aan te passen.

Resultaat combineren

Na het verwerken van alle blokken worden de eindwaarden A t/m E samengevoegd tot één hash van 160 bits.



Secure Hash Algorithm (SHA-1) Diagram

Voordelen en nadelen van SHA-1:

Voordelen van SHA-1

Snelle berekening

SHA-1 is relatief snel en efficiënt, wat het geschikt maakt voor systemen met beperkte rekenkracht.

Breed ondersteund

SHA-1 wordt nog steeds door veel systemen, protocollen en programmeertalen ondersteund vanwege het lange gebruik in de praktijk.

Vaste outputlengte

Ongeacht de invoer levert SHA-1 altijd een hash van 160 bits (40 tekens), wat handig is voor verificatie en opslag.

Nadelen van SHA-1

Kwetsbaar voor collisies

Onderzoekers hebben bewezen dat het mogelijk is om twee verschillende invoeren te maken met dezelfde hash (collisie), wat een groot beveiligingsrisico vormt.

Verouderd en onveilig voor moderne beveiliging

SHA-1 voldoet niet meer aan de huidige beveiligingsnormen en wordt niet meer aanbevolen voor toepassingen zoals digitale handtekeningen, SSL-certificaten of wachtwoorden.

Gevoelig voor aanvallen met moderne rekenkracht

Door de kracht van moderne computers en cloudsysteem is het relatief goedkoop geworden om aanvallen op SHA-1 uit te voeren.

Conclusie

SHA-1 was jarenlang een belangrijke standaard in cybersecurity, maar is nu verouderd en kwetsbaar. Voor gevoelige gegevens is het beter om veiligere algoritmes te gebruiken, zoals SHA-256 of SHA-3. Die bieden meer bescherming tegen moderne aanvallen en worden daarom vaker gebruikt in nieuwe systemen.

3)SHA-256 (Secure Hash Algorithm 256-bit) hash-algoritme

SHA-256 behoort tot de SHA-2 familie. Het is ontworpen door de Amerikaanse overheidsinstantie NSA en wordt veel gebruikt in moderne cybersecurity. SHA-256 zet een invoer (zoals een wachtwoord of bestand) om in een unieke hash van 256 bits (64 hexadecimale tekens). In tegenstelling tot oudere algoritmes zoals MD5 en SHA-1, is SHA-256 op dit moment veel veiliger en niet kwetsbaar voor bekende collisie-aanvallen. Het wordt daarom gebruikt in allerlei toepassingen, zoals blockchaintechnologie (bijvoorbeeld Bitcoin), digitale handtekeningen, SSL-certificaten en het beveiligen van wachtwoorden. SHA-256 is iets trager dan oudere methodes, maar biedt veel sterkere beveiliging, wat het zeer geschikt maakt voor gevoelige data en moderne systemen.

Hoe werkt SHA-256?

Invoer voorbereiden (padding)

De originele data wordt eerst aangevuld (gepadd) zodat de totale lengte een veelvoud is van 512 bits. Dit gebeurt op een vaste manier en bevat ook een vermelding van de oorspronkelijke lengte.

Verdelen in blokken

De gepadde data wordt opgedeeld in blokken van 512 bits.

Initialiseren van registers

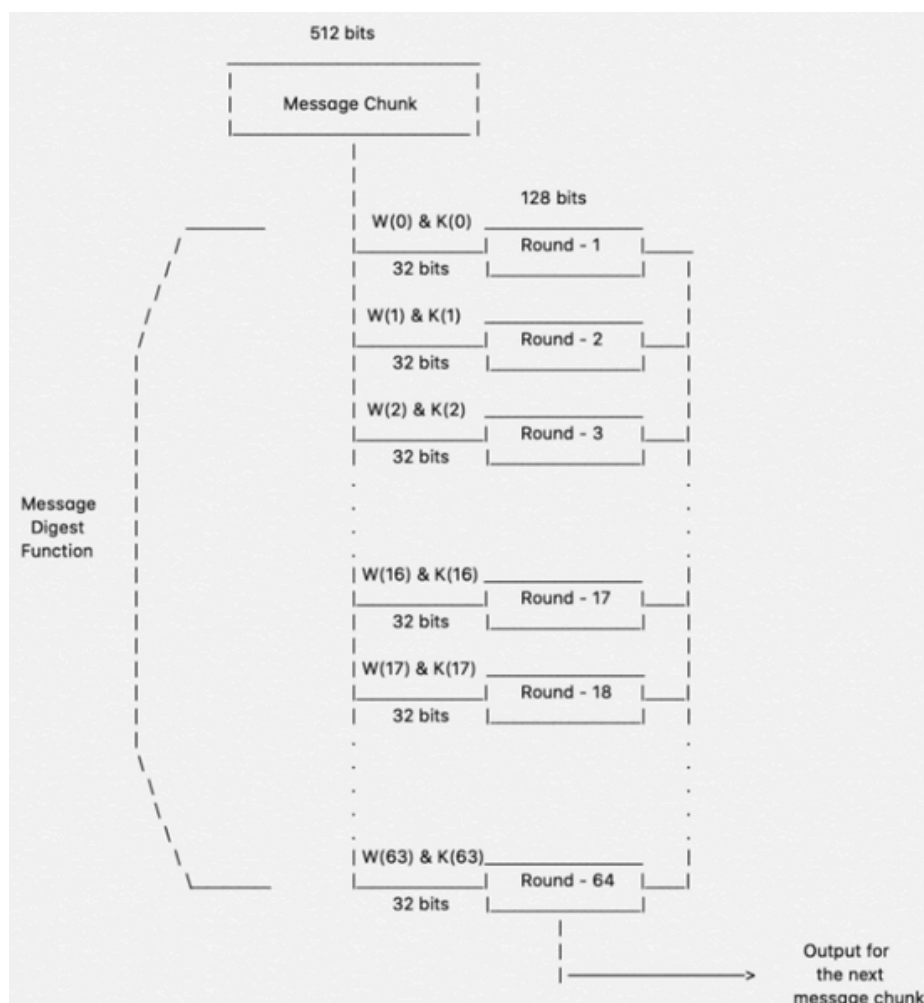
SHA-256 gebruikt acht vaste beginwaarden (H0 t/m H7), elk 32 bits lang. Deze worden telkens aangepast tijdens de verwerking.

Verwerking per blok (64 ronden)

Elk blok wordt verwerkt in 64 bewerkingsrondes. Hierbij worden logische functies gebruikt (zoals XOR, AND, ROTR), samen met constante waarden en wiskundige operaties. Elke ronde verandert de interne registers.

Samenvoegen van resultaten

Na het verwerken van alle blokken worden de acht eindwaarden samengevoegd tot één uiteindelijke hash van 256 bits (64 hexadecimale tekens).



Voordelen en nadelen van SHA-256:

Voordelen van SHA-256

Zeer veilig

SHA-256 is op dit moment niet kwetsbaar voor collisies of andere bekende aanvallen.

Betrouwbaar voor gevoelige data

Wordt gebruikt in beveiligde systemen zoals blockchain, SSL, en wachtwoordopslag.

Vaste output van 256 bits

Levert altijd een hash van 64 hexadecimale tekens, ongeacht de invoer.

Veel gebruikt en goed ondersteund

Ondersteund in bijna alle moderne programmeertalen en beveiligingsprotocollen.

Nadelen van SHA-256

Langzamer dan oudere algoritmes

Door de complexiteit is SHA-256 trager dan MD5 of SHA-1, vooral bij grote hoeveelheden data.

Meer rekenkracht nodig

Verbruikt meer CPU en energie, wat minder geschikt kan zijn voor lichte apparaten.

Conclusie:

SHA-256 is op dit moment één van de veiligste en meest gebruikte hash-algoritmes. Het is veel sterker dan oudere methodes zoals MD5 en SHA-1, en wordt daarom gebruikt in toepassingen waar veiligheid belangrijk is, zoals wachtwoordbeveiliging, blockchain en digitale handtekeningen. Hoewel het iets trager is, weegt de sterke beveiliging daar ruimschoots tegenop.

4)bcrypt hash-algoritme

bcrypt is een modern en veilig algoritme dat speciaal is ontworpen voor het hashen van wachtwoorden. In tegenstelling tot oudere hashfuncties zoals MD5 of SHA-1, is bcrypt langzaam en rekent bewust traag, zodat het moeilijker wordt voor hackers om miljoenen wachtwoorden snel te proberen via brute-force aanvallen.

bcrypt gebruikt ook automatisch een salt (willekeurige extra data), wat ervoor zorgt dat twee dezelfde wachtwoorden toch een andere hash krijgen. Hierdoor zijn rainbow table-aanvallen vrijwel onmogelijk.

bcrypt is vandaag de dag één van de veiligste keuzes voor het opslaan van wachtwoorden in databases. Het wordt veel gebruikt in moderne systemen, websites en applicaties.

Hoe werkt bcrypt?

Salt genereren

bcrypt maakt automatisch een unieke salt aan (extra willekeurige data). Hierdoor krijgt elk wachtwoord een andere hash, zelfs als twee mensen hetzelfde wachtwoord gebruiken.

Hash berekenen met kostenfactor

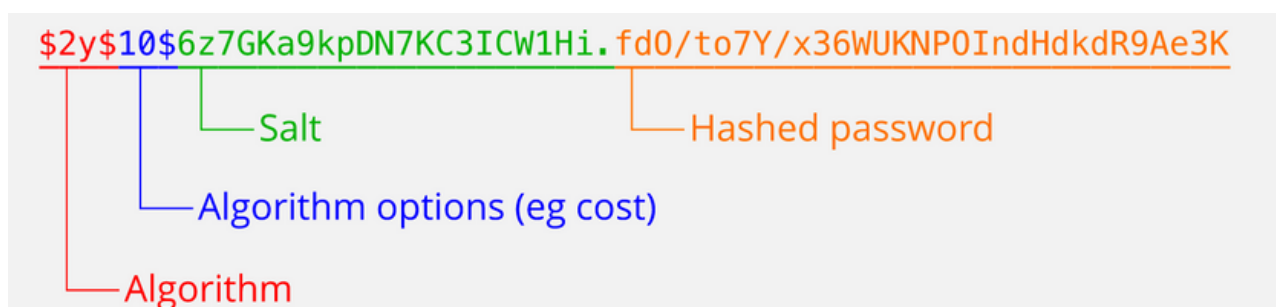
bcrypt voert de hashbewerking meerdere keren uit. Hoe vaker, hoe trager het proces. Dit wordt bepaald door een kostenfactor (bijvoorbeeld 10 of 12). Hoe hoger dit getal, hoe veiliger – maar ook hoe trager.

Resultaat: veilige hash

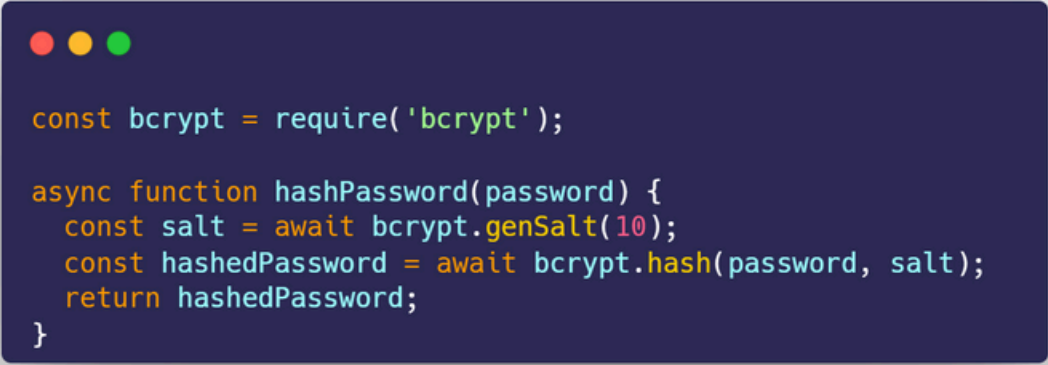
Het resultaat is een hash die de salt bevat én informatie over de kostenfactor. Zo kan bij controle van het wachtwoord dezelfde bewerking opnieuw worden gedaan.

Wachtwoordcontrole

Wanneer iemand inlogt, wordt het ingevoerde wachtwoord opnieuw gehasht met dezelfde salt en kosten. bcrypt vergelijkt dit resultaat met de opgeslagen hash.



bcrypt Algorithm



```
const bcrypt = require('bcrypt');

async function hashPassword(password) {
  const salt = await bcrypt.genSalt(10);
  const hashedPassword = await bcrypt.hash(password, salt);
  return hashedPassword;
}
```

bcrypt Js code

voordelen en nadelen van bcrypt:

Voordelen van bcrypt

Zeer veilig voor wachtwoorden

bcrypt is speciaal gemaakt om wachtwoorden te beveiligen en is op dit moment één van de veiligste methodes.

Maakt gebruik van salt

Elk wachtwoord krijgt automatisch een unieke salt, waardoor dezelfde wachtwoorden toch een andere hash opleveren.

Instelbare traagheid (kostenfactor)

Hoe hoger de kostenfactor, hoe trager het algoritme — en dus moeilijker voor hackers om brute-force toe te passen.

Bescherming tegen rainbow table-aanvallen

Door de salt zijn vooraf gemaakte hash-lijsten (rainbow tables) nutteloos.

Nadelen van bcrypt

Langzamer dan andere hash-algoritmes

bcrypt is bewust traag, wat goed is voor veiligheid, maar minder efficiënt bij grote hoeveelheden data.

Meer CPU-verbruik

Door de berekeningen kan bcrypt zwaarder zijn voor systemen met beperkte rekenkracht, zoals oudere servers of mobiele apparaten.

Conclusie:

bcrypt is een uitstekende keuze voor het beveiligen van wachtwoorden dankzij zijn trage werking, ingebouwde salt en sterke bescherming tegen aanvallen.

5) Argon2 hash-algoritme

Argon2 is een modern en zeer veilig algoritme voor het hashen van wachtwoorden. Het is in 2015 uitgeroepen tot winnaar van de internationale Password Hashing Competition (PHC) en wordt gezien als de opvolger van oudere algoritmes zoals bcrypt en PBKDF2. Argon2 is speciaal ontworpen om goed te werken tegen moderne aanvallen, zoals brute-force en GPU-aanvallen. Wat Argon2 bijzonder maakt, is dat het niet alleen veel rekenkracht gebruikt, maar ook veel geheugen (RAM). Dit maakt het extra moeilijk voor hackers om grote aantallen wachtwoorden tegelijk te kraken, zelfs met krachtige computers. Er zijn drie varianten: Argon2d, Argon2i en Argon2id, elk met een iets andere focus op veiligheid. Tegenwoordig wordt Argon2 steeds meer gebruikt in moderne systemen en toepassingen waar hoge wachtwoordbeveiliging belangrijk is.

Hoe werkt Argon2?

Belangrijkste stappen:

Salt toevoegen

Net als bcrypt gebruikt Argon2 een salt: een willekeurig stukje data dat aan het wachtwoord wordt toegevoegd. Hierdoor krijgen dezelfde wachtwoorden toch een andere hash.

Gebruik van geheugen (RAM)

Argon2 gebruikt veel geheugen bij het berekenen van de hash. Dit maakt het lastig voor aanvallers om met speciale hardware snel wachtwoorden te kraken.

Instelbare sterkte

Argon2 laat je kiezen hoeveel tijd, geheugen en aantal keren herhalen het algoritme moet gebruiken. Zo kun je de beveiliging aanpassen aan je systeem.

Resultaat: veilige hash

Het eindresultaat is een unieke, veilige hash die moeilijk terug te draaien is, zelfs met brute-force aanvallen.

Argon2 heeft drie varianten:

- Argon2d: sterk tegen GPU-aanvallen, gebruikt data-afhankelijke toegang tot geheugen.
- Argon2i: veiliger tegen side-channel aanvallen, gebruikt data-onafhankelijke toegang.
- Argon2id: combinatie van beide, vaak aanbevolen voor algemeen gebruik.

Voordelen en nadelen van Argon2:

Voordelen van Argon2

Zeer veilig

Argon2 is een van de veiligste algoritmes voor het opslaan van wachtwoorden. Het is speciaal gemaakt om goed te beschermen tegen brute-force en andere aanvallen.

Gebruik van geheugen (RAM)

Argon2 gebruikt veel geheugen tijdens het hashen, wat het moeilijk maakt voor hackers om met snelle computers of GPU's veel wachtwoorden tegelijk te kraken.

Flexibel in instellingen

Je kunt zelf bepalen hoeveel geheugen, tijd en herhalingen Argon2 gebruikt. Zo kun je de balans kiezen tussen snelheid en veiligheid.

Winnaar van de Password Hashing Competition

Argon2 is getest en goedgekeurd door experts wereldwijd, wat aantoont dat het betrouwbaar is.

Nadelen van Argon2

Niet overal ondersteund

Omdat Argon2 vrij nieuw is, wordt het nog niet in alle systemen, talen of applicaties standaard ondersteund.

Zwaarder voor oudere systemen

Door het hoge gebruik van geheugen en rekenkracht kan het trager zijn op oude of zwakke apparaten.

Conclusie:

Argon2 is een moderne en krachtige manier om wachtwoorden te beveiligen. Het is veiliger dan oudere methodes zoals bcrypt, vooral tegen aanvallen met moderne hardware.

Vergelijking van hashing-algoritmes

Eigenschap	MD5	SHA-1	SHA-256	bcrypt	Argon2
Veiligheid	Zwak, makkelijk te kraken	Verouderd en kwetsbaar	Sterk	Zeer veilig	Zeer veilig
Output lengte	128 bits (32 tekens)	160 bits (40 tekens)	256 bits (64 tekens)	192+ bits (afhankelijk)	256+ bits (instelbaar)
Gebruik van salt	Nee	Nee	Nee	Ja, automatisch	Ja, standaard
Traagheid	snel	snel	snel	Traag	Traag
Aanpasbare sterkte	Nee	Nee	Nee	Ja, via kostenfactor	Ja, via tijd, geheugen en herhalingen
Bescherming tegen GPU-aanval	Geen	Geen	Beperkt	Goed	Uitstekend
Aanbevolen voor wachtwoorden	Nee	Nee	Nee	Ja	Beste keuze
Toepassingen	Oude systemen, checksums	Oude software	SSL, blockchain, digitale handtekeningen	SSL, blockchain, digitale handtekeningen	Moderne systemen, apps, databases

Wachtwoorden Achterhalen

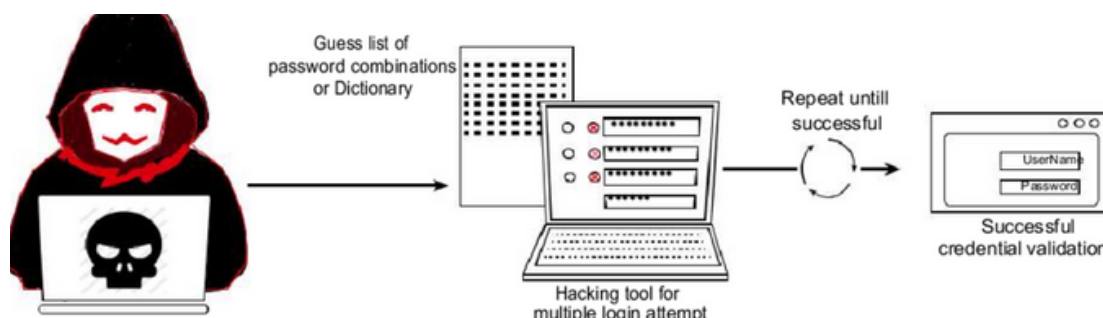
Wachtwoorden zijn de eerste verdedigingslinie voor toegang tot persoonlijke gegevens en systemen. Toch zijn veel wachtwoorden kwetsbaar en kunnen ze worden achterhaald met verschillende technieken. Denk hierbij aan brute-force aanvallen, dictionary-aanvallen of het misbruiken van zwakke encryptie. Het doel van dit onderdeel is om inzicht te krijgen in hoe wachtwoorden worden gekraakt, met welke tools dit gebeurt en hoe men zich hiertegen kan beschermen.

Disclaimer

De informatie en tools die in dit document worden besproken, zijn uitsluitend bedoeld voor educatieve en ethische doeleinden. Het doel is om inzicht te krijgen in de werking van wachtwoordbeveiliging en hoe systemen beter beveiligd kunnen worden.

Basisprincipe van Wachtwoordkraken

Een wachtwoordkraken is een methode De hacker gebruikt een programma om elke mogelijke combinatie van tekens te proberen om het wachtwoord te raden. Omdat de hash een vaste string is, moet de hacker verschillende combinaties genereren, deze hashen en vergelijken met de hash in de database. Als de gegenereerde hash overeenkomt, heeft de hacker het wachtwoord gevonden. Hoe complexer het wachtwoord, hoe langer het duurt om het te kraken. Brute force-aanvallen zijn tijdrovend, maar effectief bij zwakkere wachtwoorden of onveilige hashingalgoritmen.



Methodes om Wachtwoorden te Kraken

1.Brute Force-aanval

Bij een Brute Force-aanval probeert een hacker een wachtwoord te raden door alle mogelijke combinaties van tekens (letters, cijfers, symbolen) uit te proberen. Dit kan lang duren, vooral bij lange en complexe wachtwoorden, maar met krachtige computers is het soms mogelijk.

2.Dictionary-aanval

Een Dictionary-aanval gebruikt een lijst van veelvoorkomende wachtwoorden (zoals "wachtwoord123", "admin" of "welkom01") om in te loggen. Hackers maken vaak gebruik van gelekte wachtwoorden uit eerdere datalekken om hun kans te vergroten

3.Rainbow Table-aanval

Wachtwoorden worden vaak opgeslagen als hashes (versleutelde versies). Een Rainbow Table bevat vooraf berekende hashes van veelgebruikte wachtwoorden. Als een hacker toegang krijgt tot een database met hashes, kan hij deze tabel gebruiken om het originele wachtwoord te vinden.

4.Keylogging

Een keylogger is malware die toetsaanslagen registreert. Als een gebruiker zijn wachtwoord intoetst, kan de hacker het zo achterhalen zonder het wachtwoord zelf te hoeven kraken.

5.Phishing & Social Engineering

Soms wordt het wachtwoord niet technisch gekraakt, maar via misleiding verkregen. Bijvoorbeeld via een neppe e-mail (phishing) of door iemand te manipuleren (social engineering) om het wachtwoord vrij te geven.

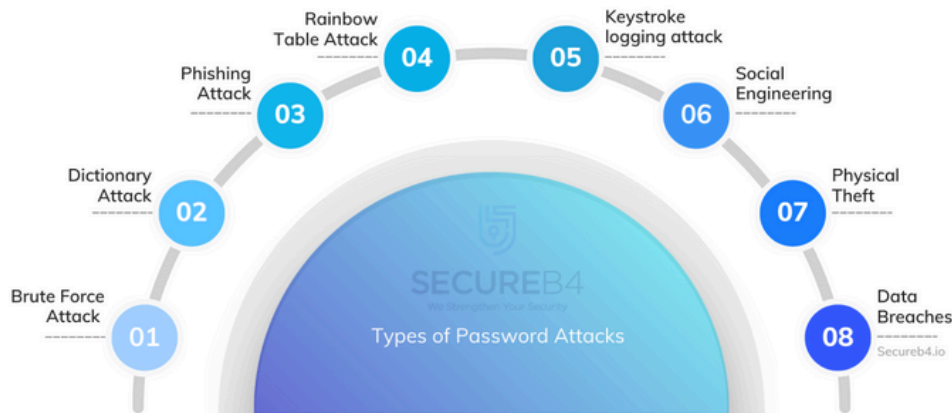
6. Credential Stuffing

Veel mensen hergebruiken wachtwoorden. Bij Credential Stuffing proberen hackers gelekte inloggegevens (bijvoorbeeld van een oud datalek) op andere websites, in de hoop dat hetzelfde wachtwoord daar ook werkt.

Different Types of Password Attacks

Here is an overview of some of the different types of password attacks commonly used by hackers

Secureb4.io



Wachtwoordkraken met Tools in Kali Linux

Naast de verschillende methodes van wachtwoordkraken, gebruiken hackers vaak gespecialiseerde tools om het proces te versnellen en te automatiseren. Kali Linux bevat een breed scala aan tools die gebruikt kunnen worden voor het kraken van wachtwoorden.

Enkele veelgebruikte tools zijn:

1.Hashcat

Hashcat is een van de krachtigste wachtwoord-crackingtools die beschikbaar is. Het is speciaal ontworpen om gehashte wachtwoorden te kraken met behulp van brute force, dictionary-aanvallen en hybride methodes. Hashcat maakt gebruik van CPU of GPU om miljoenen hashes per seconde te berekenen, waardoor het zeer efficiënt is.

2.Hydra

Hydra (ook wel THC-Hydra) is een krachtig hulpmiddel voor het uitvoeren van brute force- en dictionary-aanvallen op inlogdiensten via het netwerk. Het ondersteunt veel protocollen zoals:

- SSH
- FTP
- HTTP/HTTPS (bijv. webformulieren)
- Telnet
- RDP
- SMB
- en meer...

Hydra probeert automatisch combinaties van gebruikersnamen en wachtwoorden totdat een juiste combinatie wordt gevonden.

3. John the Ripper

John the Ripper is een krachtige tool om gehashte wachtwoorden te kraken. Het is ontworpen voor snelheid en efficiëntie, en werkt vooral goed met wachtwoordbestanden uit Linux (zoals `/etc/shadow`), Windows (NTLM), of gelekte databases.

John werkt met verschillende methodes zoals:

- Dictionary-aanvallen
- Brute force (incremental mode)
- Rule-based attacks (variaties op woorden)
- Hybrid attacks

Introductie op de Demo: Wachtwoorden Kraken met John the Ripper

In deze demo wordt stap voor stap uitgelegd hoe gehashte wachtwoorden kunnen worden gekraakt met behulp van John the Ripper in Kali Linux. De oefening is gebaseerd op een project uit de GitHub-repository [Ethical Hacking Projects for Beginners](#) en is bedoeld om praktische ervaring op te doen met wachtwoordbeveiliging en hash-analyse.

De demo bevat:

- Analyse van voorbeeldhashes
- Het uitvoeren van een woordenlijstaanval met John
- Het uitlezen van gekraakte wachtwoorden

Installatie

John the Ripper is vooraf geïnstalleerd op Kali Linux. De installatie kan worden gecontroleerd of bijgewerkt met het volgende commando:

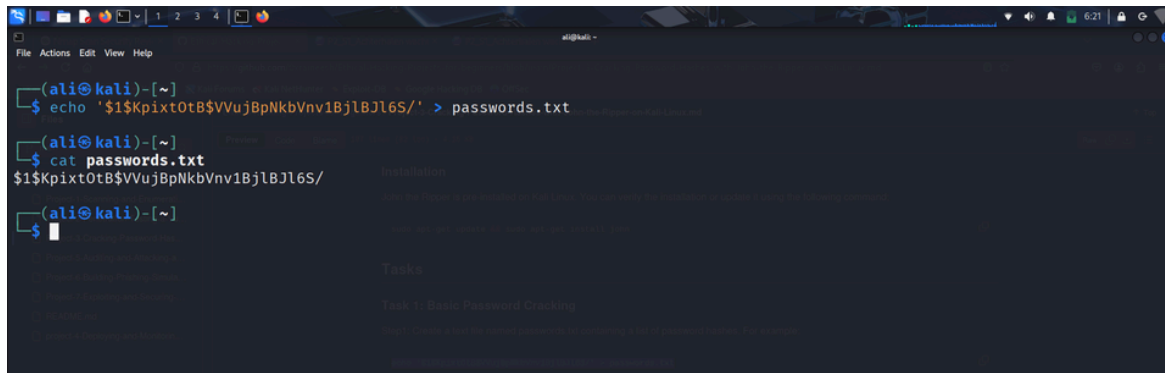
```
sudo apt-get update && sudo apt-get install john
```



Opdracht 1: Basis Wachtwoordkraken

Stap 1: Maak een tekstbestand genaamd passwords.txt met daarin een lijst van gehashte wachtwoorden. Bijvoorbeeld:

```
echo '$1$Kp1xt0tB$VVujBpNkbVnv1Bj1BJL6S/' > passwords.txt
```



```
(ali@kali)-[~]
$ echo '$1$Kp1xt0tB$VVujBpNkbVnv1Bj1BJL6S/' > passwords.txt
(ali@kali)-[~]
$ cat passwords.txt
$1$Kp1xt0tB$VVujBpNkbVnv1Bj1BJL6S/
(ali@kali)-[~]
$
```

Stap 2: Open een terminal op de Kali Linux-machine. Voer John the Ripper uit om de hashes in passwords.txt te kraken:

```
john passwords.txt
```

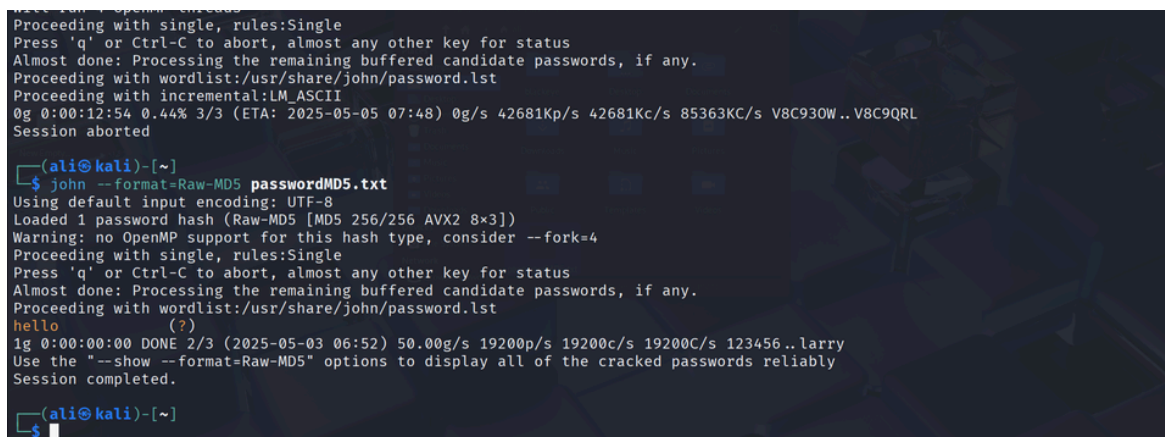
Let op:

John the Ripper kan het type hash automatisch detecteren, maar dit maakt het proces trager. Het is efficiënter om het specifieke hashtype handmatig op te geven. Gebruik hiervoor een tool zoals hash-identifier om het type hash vooraf te bepalen:

```
hash-identifier
```

Voer de hash in en laat het algoritme herkennen. Gebruik vervolgens de juiste --format= optie bij het John-commando voor snellere resultaten.

```
john --format=md5crypt passwords.tx
```



```
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:LM_ASCII
0g 0:00:12:54 0.44% 3/3 (ETA: 2025-05-05 07:48) 0g/s 42681Kp/s 42681Kc/s 85363KC/s V8C930W..V8C9QRL
Session aborted

(ali@kali)-[~]
$ john --format=Raw-MD5 passwordMD5.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
hello (?)
1g 0:00:00:00 DONE 2/3 (2025-05-03 06:52) 50.00g/s 19200p/s 19200c/s 19200C/s 123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(ali@kali)-[~]
$
```

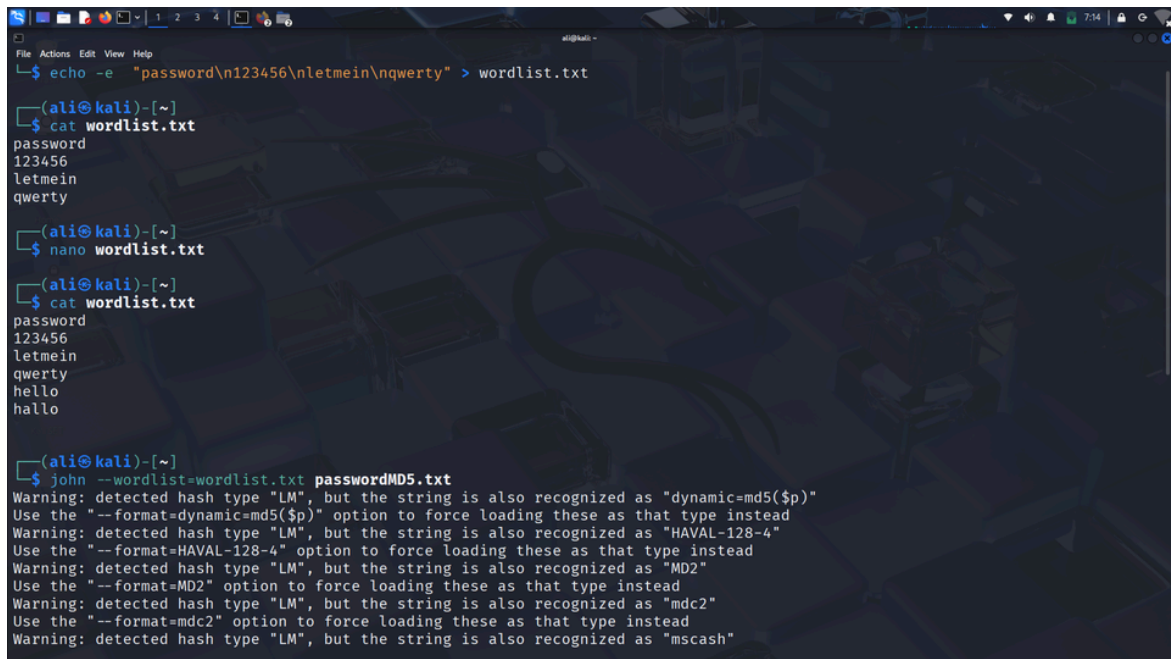
Opdracht 2: Gebruik van een Aangepaste Woordenlijst(Dictionary-aanval)

Stap 1: Maak een aangepaste woordenlijst met potentiële wachtwoorden in een bestand genaamd wordlist.txt. Bijvoorbeeld:

```
echo -e "password\n123456\nletmein\nqwerty" > wordlist.txt
```

Stap 2: Gebruik John the Ripper met deze aangepaste woordenlijst om de wachtwoordhashes in passwords.txt te kraken:

```
john --wordlist=wordlist.txt --format=Raw-md5 passwords.txt
```



```
(ali@kali)-[~]
└─$ echo -e "password\n123456\nletmein\nqwerty" > wordlist.txt

(ali@kali)-[~]
└─$ cat wordlist.txt
password
123456
letmein
qwerty

(ali@kali)-[~]
└─$ nano wordlist.txt

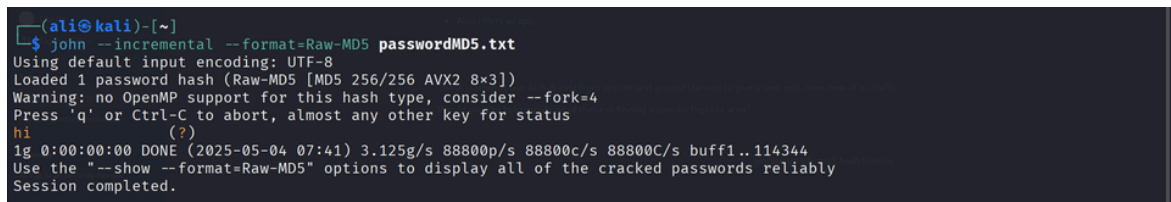
(ali@kali)-[~]
└─$ cat wordlist.txt
password
123456
letmein
qwerty
hello
hallo

(ali@kali)-[~]
└─$ john --wordlist=wordlist.txt passwordMD5.txt
Warning: detected hash type "LM", but the string is also recognized as "dynamic-md5($p)"
Use the "--format=dynamic-md5($p)" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "HAVAL-128-4"
Use the "--format=HAVAL-128-4" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "MD2"
Use the "--format=MD2" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "mdc2"
Use the "--format=mdc2" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "mscash"
```

Opdracht 3: Brute-force aanval (incremental mode)

Bij een brute-force aanval probeert John alle mogelijke combinaties van tekens totdat de juiste wordt gevonden. Dit duurt langer, maar vereist geen woordenlijst.

```
john --incremental --format=md5crypt passwords.txt
```



```
(ali@kali)-[~]
└─$ john --incremental --format=Raw-MD5 passwordMD5.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
hi (7)
1g 0:00:00:00 DONE (2025-05-04 07:41) 3.125g/s 88800p/s 88800c/s 88800C/s buff1..114344
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Uitdagingen met John the Ripper

Tijdens het kraken gebruikt John the Ripper standaard 100% van de CPU en benut alle beschikbare threads om de snelheid te maximaliseren. Dit is normaal gedrag, maar het systeem kan hierdoor tijdelijk trager aanvoelen. Er zijn enkele opties beschikbaar om het resourcegebruik beter te beheren:

```
--fork=<aantal>
```

Hiermee wordt John gestart met een specifiek aantal parallele processen (cores). Bijvoorbeeld:

```
john --fork=4 --format=md5crypt passwords.txt
```

```
--save-memory=1
```

Hiermee wordt John gevraagd om minder geheugen te gebruiken, wat nuttig is op systemen met beperkte RAM. Bijvoorbeeld:

```
john --format=md5crypt --save-memory=1 passwords.txt
```

Conclusie

John the Ripper is een krachtige tool voor het kraken van wachtwoorden. Het ondersteunt verschillende aanvalsmethoden zoals dictionary-, brute-force- en single-crack-aanvallen. Dankzij de ondersteuning voor vele hashtypes, snelheid, en uitbreidbaarheid is John geschikt voor zowel beginners als gevorderde gebruikers in cybersecurity.

Deze tool is gekozen omdat:

- Het standaard is inbegrepen in Kali Linux
- Het open-source en actief onderhouden is
- Het duidelijke output en controle biedt over het proces

Introductie op de Demo: Wachtwoorden Kraken met Hashcat

In deze demo wordt getoond hoe wachtwoorden kunnen worden gekraakt met Hashcat, een krachtige GPU-CPU gebaseerde tool voor het kraken van gehashte wachtwoorden. Hashcat ondersteunt meerdere aanvalsmethoden zoals dictionary-aanvallen, combinaties, brute-force en maskers.

De demo laat zien hoe:

- Een hashbestand wordt voorbereid
- Hashcat wordt gebruikt met een woordenlijst
- Het kraken efficiënt verloopt met behulp van de GPU

Opdracht 1: Basis Wachtwoordkraken met Hashcat

Stap 1: Maak een bestand aan met een gehashte waarde. Bijvoorbeeld:

```
echo '49f68a5c8493ec2c0bf489821c21fc3b' > hashes.txt
```

Stap 2: Start Hashcat met een eenvoudige woordenlijstaanval. Gebruik bijvoorbeeld de bekende woordenlijst rockyou.txt:

```
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

Uitleg van de opties:

- -m = Hash-type (MD5 Crypt)
- -a 0 = Aanvalstype: dictionary mode
- hashes.txt = Bestand met de hash
- /usr/share/wordlists/rockyou.txt = Woordenlijst

Opmerking

1.het volgende commando om alle beschikbare hashmodulen en opties van Hashcat te bekijken:

```
hashcat --help
```

Hiermee verschijnt een lijst met alle ondersteunde hashmodulen (zoals SHA1, MD5, bcrypt, NTLM, enz.) inclusief het juiste -m nummer dat nodig is voor elk type hash. Dit is handig om het juiste formaat te kiezen voordat een aanval wordt gestart.

2.Gebruik een tool zoals hash-identifier om het type hash te achterhalen voordat je Hashcat gebruikt. Dit helpt om het juiste -m nummer te kiezen en voorkomt fouten of vertraging.

```
hash-identifier 49f68a5c8493ec2c0bf489821c21fc3b
```


Opdracht 2: Brute-force Aanval met Hashcat

```
hashcat -m 0 -a 3 hashes.txt ?l?l?l?l
```

Uitleg:

- -m 0 = MD5 Crypt
- -a 3 = Aanvalstype: brute-force (mask mode)
- ?l?l?l?l = patroon van vier kleine letters (bijv. abcd, qwer)

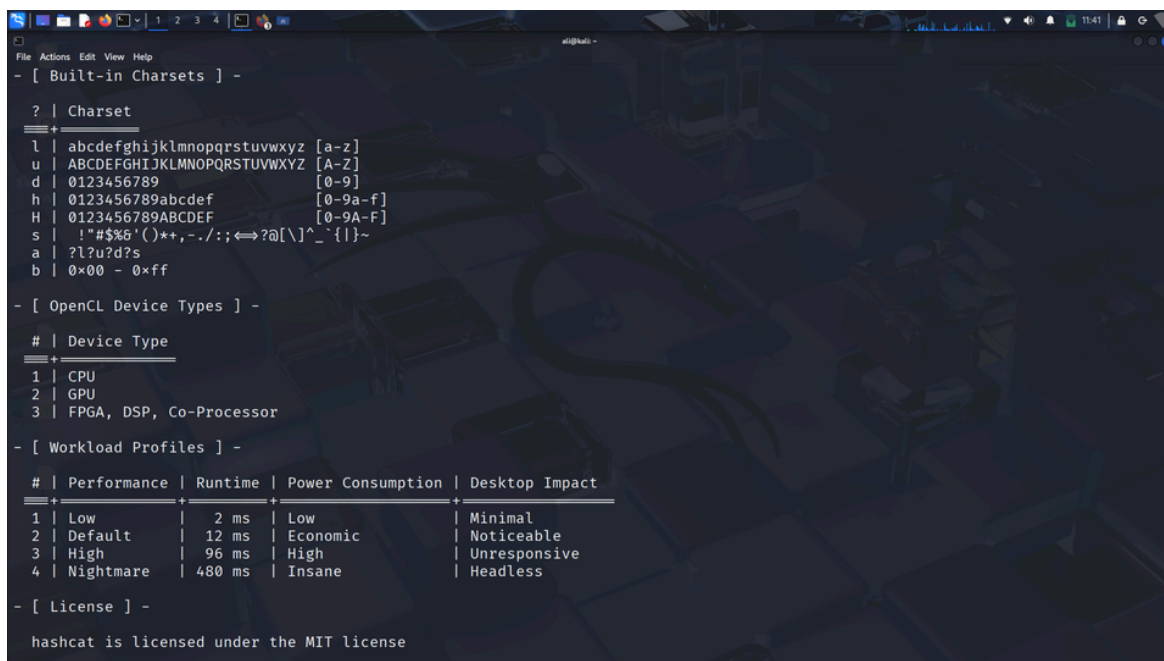
Wat betekent ?l?l?l?l in Hashcat?

Bij het uitvoeren van een brute-force aanval in Hashcat kun je gebruik maken van maskers om het zoekproces slimmer te maken. In plaats van willekeurig alle combinaties te testen, geef je met een patroon precies aan welke soorten tekens op elke positie mogen voorkomen

Dit maakt brute-force aanvallen sneller en gericht.

Een masker bestaat uit speciale tekens die elk een bepaalde tekenklasse voorstellen. Bijvoorbeeld:

- ?l kleine letters (a-z)
- ?u hoofdletters (A-Z)
- ?d cijfers (0-9)
- ?s symbolen (!@# etc.)
- ?a alle mogelijke tekens
- ?b alle bytes (0x00-0xff)



```
File Actions Edit View Help
- [ Built-in Charsets ] -

? | Charset
=====+=====
l | abcdefghijklmnopqrstuvwxyz [a-z]
u | ABCDEFGHIJKLMNOPQRSTUVWXYZ [A-Z]
d | 0123456789 [0-9]
h | 0123456789abcdef [0-9a-f]
H | 0123456789ABCDEF [0-9A-F]
s | !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
a | ?l?u?d?s
b | 0x00 - 0xff

- [ OpenCL Device Types ] -

# | Device Type
=====+=====
1 | CPU
2 | GPU
3 | FPGA, DSP, Co-Processor

- [ Workload Profiles ] -

# | Performance | Runtime | Power Consumption | Desktop Impact
=====+=====+=====+=====+=====
1 | Low | 2 ms | Low | Minimal
2 | Default | 12 ms | Economic | Noticeable
3 | High | 96 ms | High | Unresponsive
4 | Nightmare | 480 ms | Insane | Headless

- [ License ] -

hashcat is licensed under the MIT license
```

Uitdagingen met Hashcat

1. Tijdens het uitvoeren van de aanvallen ontdekte ik dat Hashcat standaard alleen de CPU gebruikte, terwijl mijn GPU niet werd herkend. Dit resulteerde in langzamere prestaties dan verwacht, aangezien Hashcat juist bekend staat om zijn snelheid via GPU-versnelling.

```
ali@kali:~$ hashcat -I
hashcat (v6.2.6) starting in backend information mode

OpenCL Info:

OpenCL Platform ID #1
Vendor..: The pocl project
Name....: Portable Computing Language
Version.: OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, LLVM 18.1.8, SLEEP, DISTRO, POCL_DEBUG

Backend Device ID #1
Type.....: CPU
Vendor.ID...: 128
Vendor.....: GenuineIntel
Name.....: cpu-haswell-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz
Version.....: OpenCL 3.0 PoCL HSTR: cpu-x86_64-pc-linux-gnu-haswell
Processor(s) ...: 4
Clock.....: 3000
Memory.Total...: 5647 MB (limited to 1024 MB allocatable in one block)
Memory.Free....: 2791 MB
Local.Memory...: 256 KB
OpenCL.Version.: OpenCL C 1.2 PoCL
Driver.Version.: 6.0+debian
```

```
ali@kali:~$ hashcat -I
hashcat (v6.2.6) starting in backend information mode

Attack- | Hash- |
Mode    | Type  | Example command
-----+-----+-----
Wordlist | $P$   | hashcat -a 0 -m 400 example400.hash example.dict
Wordlist + Rules | MD5   | hashcat -a 0 -m 0 example0.hash example.dict -r rules/best64.rule
Brute-Force | MD5   | hashcat -a 3 -m 0 example0.hash ?a?a?a?a?a
Combinator | MD5   | hashcat -a 1 -m 0 example0.hash example.dict example.dict
Association | $1$   | hashcat -a 9 -m 500 example500.hash 1word.dict -r rules/best64.rule

If you still have no idea what just happened, try the following pages:
* https://hashcat.net/wiki/howtos_videos_papers_articles_etc_in_the_wild
* https://hashcat.net/faq/

If you think you need help by a real human come to the hashcat Discord:
* https://hashcat.net/discord

ali@kali:~$ hashcat -d 2 -m 0 -a 0 passwordMD5.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

No devices found/left.

Started: Mon May 5 11:15:07 2025
Stopped: Mon May 5 11:15:07 2025
```

Na enig onderzoek ontdekte ik dat er bepaalde configuraties nodig zijn zodat Hashcat de GPU correct herkent. Helaas is het me nog niet gelukt om de juiste configuratie te vinden om mijn GPU succesvol aan Hashcat te koppelen.

2. Een tweede probleem dat ik tegenkwam, is dat Hashcat soms het proces voortijdig stopzet vanwege een gebrek aan geheugen. Dit kan gebeuren bij grote woordenlijsten of complexe hashtypes die veel rekenkracht en RAM/VRAM vereisen.

Mogelijke oplossingen zijn:

1. Beperk geheugenverbruik met

```
--optimized-kernel-enable
```

```
hashcat -m 500 -a 0 hashes.txt wordlist.txt --optimized-kernel-enable
```

Dit gebruikt een geoptimaliseerde kernel voor snellere en lichtere verwerking

2. Gebruik --segment-size om RAM-gebruik te beperken (voor oudere systemen)

```
hashcat -m 500 -a 0 hashes.txt wordlist.txt --segment-size=32
```

Hiermee gebruik je bijvoorbeeld slechts 32 MB segmenten per thread.

Conclusie: Hashcat

Hashcat is één van de snelste en meest krachtige tools voor het kraken van wachtwoordhashes. Dankzij ondersteuning voor GPU-versnelling, een breed scala aan hash-algoritmes en flexibele aanvalsmethoden (zoals woordenlijsten, brute-force en maskers)

De tool is gekozen vanwege:

- Hoge prestaties bij complexe berekeningen
- Ondersteuning voor honderden hashtypes
- Veel aanpasbare opties en strategieën
- Actieve community en uitgebreide documentatie

Proces Infographic



Stap 1 Toestemming Regelen

- Zorg voor schriftelijke goedkeuring van de eigenaar.
- Bepaal duidelijk welke systemen/hashtypes je mag testen.
- Houd rekening met privacywetten zoals de AVG.

Stap 2 Informatie Verzamelen

- Identificeer het hash-type (MD5, SHA-1, bcrypt, etc.).
- Check of er een salt wordt gebruikt voor extra beveiliging.
- Verzamel of maak een goede woordenlijst (bijv. rockyou.txt).



Stap 3 Doel Bepalen

- Kies een aanvalsmethode: brute-force, woordenboekaanval, of rainbow tables.
- Controleer of er wachtwoordbeleid is (lockout, complexiteit).

Stap 4 Tools Selecteren

- Hashcat (snelste voor GPU-kraken)
- John the Ripper (flexibel, veel hash-types)
- Hydra (online aanvallen op SSH, FTP, websites)



Stap 5 Aanval Uitvoeren

- Voer eerste tests uit in een gecontroleerde omgeving.
- Meet snelheid en aanpassingen voor optimale resultaten.

Stap 6 Resultaten Analyseren

- Noteer zwakke wachtwoorden en veelgemaakte fouten.
- Onderzoek voor betere beveiliging (sterkere hashtypes, MFA).



Best practices voor het gebruik van wachtwoorden

Na het verkrijgen van algemene informatie over de verschillende soorten hash-algoritmes en het leren kennen van krachtige tools voor wachtwoordkraken zoals Hashcat, John the Ripper en Hydra, wordt het duidelijk hoe kwetsbaar zwakke wachtwoorden kunnen zijn. Door inzicht te krijgen in de methodes van aanvallers, wordt het belang van sterke en goed beheerde wachtwoorden nóg duidelijker. Daarom is het essentieel om te weten hoe je veilige wachtwoorden kiest en beheert. In dit onderdeel worden de beste praktijken besproken om jezelf te beschermen tegen brute-force aanvallen, woordenlijstaanvallen en andere vormen van wachtwoordkraken.

Hoe maak je een sterk wachtwoord?

Gebruik lengte boven alles

Hoe langer het wachtwoord, hoe moeilijker te kraken. Streef naar minimaal 12 tot 16 tekens.

Combineer verschillende soorten tekens

Gebruik een mix van:

Hoofdletters (A–Z)

Kleine letters (a–z)

Cijfers (0–9)

Speciale tekens (!@#%^&*)

Gebruik geen voorspelbare patronen

vermijd namen, geboortedata, "123456", "qwerty" of andere bekende woorden.

Gebruik een wachtzin (passphrase)

Bijvoorbeeld: FietsenIn#DeRegen!2024 is sterk én goed te onthouden.

Gebruik per dienst een uniek wachtwoord

Hergebruik van wachtwoorden betekent dat één lek meteen meerdere accounts in gevaar brengt.

Aanvullende Tips

Gebruik een wachtwoordmanager

Bijvoorbeeld Bitwarden, KeePass of 1Password om complexe en unieke wachtwoorden te beheren.

Schakel waar mogelijk tweefactorauthenticatie (2FA) in

Zo blijft je account beschermd, zelfs als het wachtwoord wordt gekraakt.

Verander wachtwoorden regelmatig, vooral na een datalek.

Bronnenlijst

OWASP Foundation. Password Storage Cheat Sheet Informatie over veilige opslag van wachtwoorden en aanbevolen hash-algoritmes.

[https://cheatsheetseries.owasp.org/cheatsheets/Password Storage Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)

Password Hashing Competition (PHC). (2015). [Argon2 Specification](#) Officiële site van de wedstrijd waarin Argon2 als winnaar is gekozen.

Wikipedia. (n.d.). Voor basisinformatie over de verschillende algoritmes.

[MD5](#)

[SHA-1](#)

[SHA-2 \(SHA-256\)](#)

[bcrypt](#)

[Argon2](#)

Crypto101.io. [Crypto 101 – A free introductory book on cryptography](#).

Een toegankelijke uitleg over cryptografische technieken, waaronder hashing.

Hashcat Documentation

<https://hashcat.net/wiki/>

Hashcat Tool in Kali Linux

<https://www.geeksforgeeks.org/hashcat-tool-in-kali-linux/>

John the Ripper GitHub-pagina

<https://github.com/openwall/john>

Cybersecurity Guide – How John the Ripper Works

<https://www.geeksforgeeks.org/how-to-use-john-the-ripper-in-kali-linux/>

Cybersecurity & Infrastructure Security Agency (CISA) – Password Tips

https://www.cisa.gov/sites/default/files/publications/NCSAM_CreatingPasswords_2020.pdf

Bitwarden – Password Security Guide

<https://bitwarden.com/blog/3-tips-from-nist-to-keep-passwords-secure/>